

PE-EC802C

VLSI Design Automation

Combined Module Notes

Modules 1 – 5

Semester VIII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: VLSI Design Flow & Abstraction Levels	4
1 VLSI Design Flow & Abstraction Levels	4
1.1 Gajski-Kuhn Y-Chart	4
1.2 Physical Design Flow Phases	5
1.3 VLSI Design Automation Tools Overview	5
2 Algorithmic Graph Theory & Complexity	6
2.1 Graph Representation Models	6
2.2 Bipartite Graphs in VLSI	6
2.3 Tractability and Complexity Classes	7
2.4 Data Structures & Algorithms in VLSI CAD	7
3 Combinatorial Optimization & Heuristic Methods	7
3.1 1. Simulated Annealing (SA)	7
3.1.1 Algorithmic Flow	8
3.2 2. Genetic Algorithms (GA)	9
3.3 3. Integer Linear Programming (ILP)	10
Module 2: Layout Compaction	14
1 Layout Compaction	14
1.1 1D vs. 2D Compaction	14
1.2 Constraint Graph Compaction	14
2 Circuit Placement	15
2.1 1. Force-Directed Placement	15
2.2 2. Min-Cut Placement (Breuer's Algorithm)	15
3 Circuit Partitioning	16
3.1 Circuit Representation	16
3.2 Kernighan-Lin (KL) Algorithm	16
3.2.1 Mathematical Formulations	16
3.2.2 Algorithmic Steps for One Pass	17
3.3 Fiduccia-Mattheyses (FM) Partitioning Algorithm	18
Module 3: Circuit Floorplanning	22
1 Circuit Floorplanning	22
1.1 Slicing vs. Non-Slicing Floorplans	22
1.2 Polish Expressions and Slicing Trees	22
1.3 Shape Functions and Sizing Optimization	23
1.4 Non-Slicing Floorplan Representations	23
2 Circuit Routing	23
2.1 Classification of Routing Problems & Methods	23

2.2	Global Routing & Algorithms	24
2.2.1	Key Global Routing Algorithms	24
2.3	Lee's Maze Routing Algorithm	24
2.3.1	Algorithmic Flow	24
2.3.2	Limitations of Lee's Router	25
2.4	Line-Probe Area Routing	25
3	Channel Routing	26
3.1	Left-Edge Algorithm	26
3.2	Horizontal and Vertical Constraint Graphs	26

Module 4: VLSI Simulation Techniques **30**

1	VLSI Simulation Techniques	30
1.1	Gate-Level Event-Driven Simulation	30
1.2	Switch-Level Simulation	30
1.3	Compiled-Code vs. Event-Driven Simulation	31
2	Combinational Logic Synthesis	31
2.1	1. Technology-Independent Optimization	31
2.2	2. Technology Mapping	32
3	Two-Level Logic Synthesis	32
3.1	Quine-McCluskey (QM) Minimisation	32
3.2	Espresso Heuristic Minimiser	32
4	Binary Decision Diagrams (BDD)	33
4.1	Shannon's Expansion Theorem	33
4.2	Reduced Ordered BDDs (ROBDD)	33
4.3	The BDD Apply Operation	34

Module 5: High-Level Synthesis (HLS) Core Concepts **38**

1	High-Level Synthesis (HLS) Core Concepts	38
1.1	Core Compilation Steps	38
1.2	Hardware & Architecture Models in HLS	38
1.3	Internal Representations	39
2	Scheduling Algorithms	39
2.1	1. ASAP Scheduling (As-Soon-As-Possible)	39
2.2	2. ALAP Scheduling (As-Late-As-Possible)	39
2.3	3. List Scheduling	40
2.4	4. Force-Directed Scheduling (FDS)	40
3	Hardware Binding & Transformations	40
3.1	The Assignment (Binding) Problem Formulation	40
3.2	1. Register Binding (Left-Edge Algorithm)	41
3.3	2. High-Level Optimisation Transformations	41

Module 1

VLSI Design Flow & Abstraction Levels

1. VLSI Design Flow & Abstraction Levels

VLSI Design Automation

VLSI Design Automation refers to the use of computer-aided design (CAD) software tools to automate the translation of a high-level description of an electronic system into a physical layout fabricated on a silicon wafer. It covers synthesis, optimization, physical layout, verification, and testing.

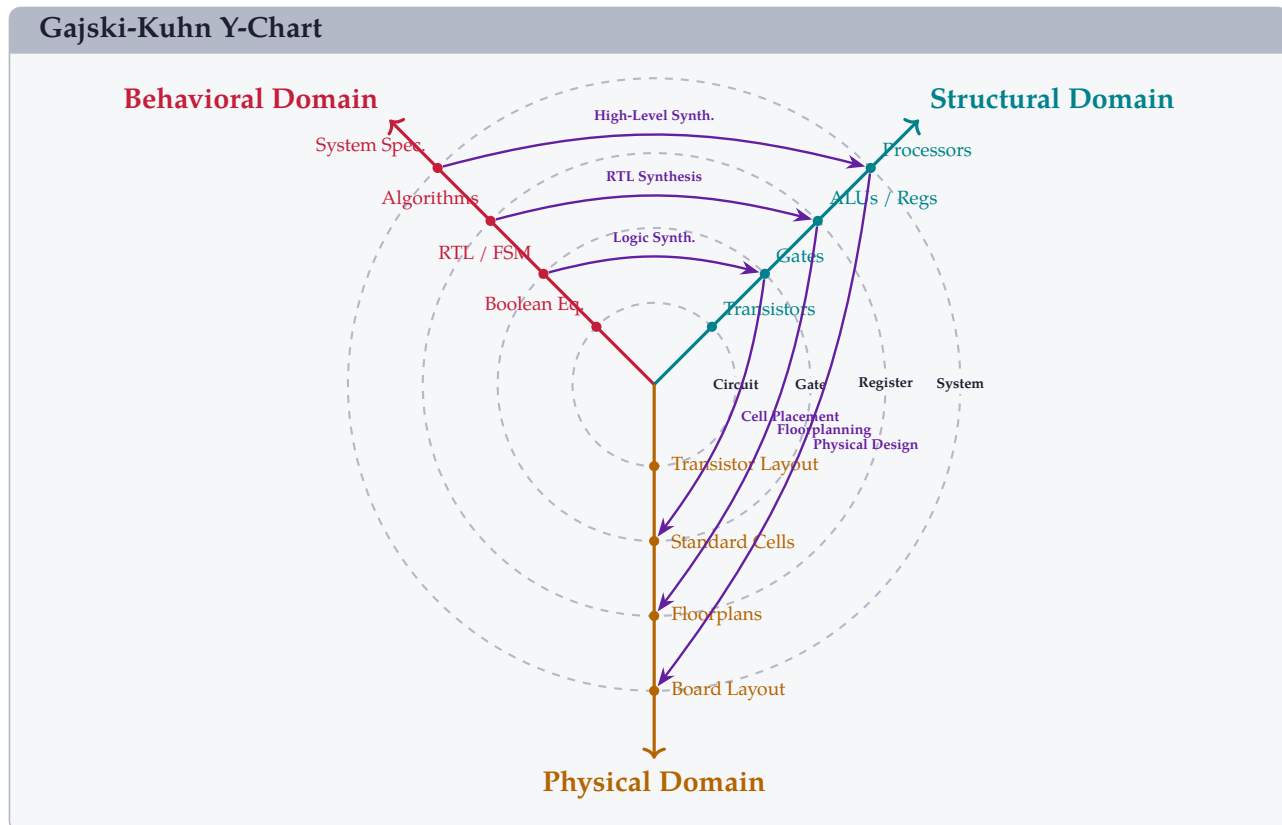
1.1 Gajski-Kuhn Y-Chart

The Gajski-Kuhn Y-Chart is a framework that describes electronic system design using three distinct domains represented as three axes of a Y:

- **Behavioral Domain:** Describes *what* the system does in terms of function, transfer functions, executable algorithms, and state machines.
- **Structural Domain:** Describes *how* the system is connected in terms of hardware entities like registers, ALUs, logic gates, and transistors.
- **Physical Domain:** Describes the physical implementation details on silicon, including floorplans, cell placement, macro routing, and transistor layouts.

The concentric rings crossing the three axes represent the **levels of abstraction**: System, Register Transfer Level (RTL), Gate/Logic, and Circuit/Transistor levels.

Gajski-Kuhn Y-Chart



1.2 Physical Design Flow Phases

Physical design is the process of converting the synthesized netlist into a geometric layout ready for fabrication. The phases include:

1. **Partitioning:** Dividing a large system into smaller, manageable blocks to minimize communication overhead and simplify design.
2. **Floorplanning & Sizing:** Positioning major blocks on the silicon die, assigning aspect ratios, and planning power/ground distribution.
3. **Placement:** Assigning exact geometric coordinates to standard cells inside the floorplanned blocks to minimize total wirelength.
4. **Routing:** Connecting standard cells and pads using metal wire tracks. This is divided into *Global Routing* (allocating rough paths) and *Detailed Routing* (assigning wires to specific metal layers and tracks).
5. **Compaction:** Shrinking the layout to its absolute minimum size while adhering to the technological design rules (spacing, width rules).

1.3 VLSI Design Automation Tools Overview

Modern VLSI design flows rely heavily on Electronic Design Automation (EDA) tools from major vendors (such as Synopsys, Cadence, and Siemens EDA) as well as open-source projects. These tools automate different stages of the design cycle:

Design Phase	Commercial / Industry Standard	Academic / Open-Source
Logic Synthesis	Synopsys Design Compiler, Cadence Genus	Yosys, ABC
Placement & Routing	Synopsys IC Compiler II, Cadence Innovus	OpenROAD, Graywolf
Physical Verification	Siemens EDA Calibre, Synopsys IC Validator	Magic, Netgen
Simulation	Synopsys VCS, Cadence Xcelium, ModelSim	Icarus Verilog, Verilator

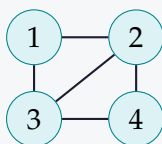
2. Algorithmic Graph Theory & Complexity

Physical design layouts are represented abstractly using graph data structures. For example, standard cells are represented as vertices (V), and the interconnect wires are represented as edges (E).

2.1 Graph Representation Models

- **Adjacency Matrix (A):** A 2D array of size $|V| \times |V|$ where $A[i][j] = 1$ if there is an edge between v_i and v_j , and 0 otherwise. Suitable for dense graphs ($|E| \approx |V|^2$).
- **Adjacency List:** An array of lists of size $|V|$, where index i stores the neighbors of vertex v_i . Memory efficient for sparse graphs ($|E| \ll |V|^2$).

Graph Representation Example



Adjacency Matrix:

	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	1
4	0	1	1	0

Adjacency List:

Node	Neighbors
1	2 → 3
2	1 → 3 → 4
3	1 → 2 → 4
4	2 → 3

2.2 Bipartite Graphs in VLSI

A graph $G = (V, E)$ is **bipartite** if its vertices can be partitioned into two disjoint sets U and W such that every edge $e \in E$ connects a vertex in U to a vertex in W (no edges exist between vertices of the same set). Bipartite graphs are used to represent:

- Pin-to-net connectivity lists in floorplanning.
- Left-Edge algorithm horizontal/vertical routing constraints.
- Hardware binding/allocation problems.

2.3 Tractability and Complexity Classes

VLSI design problems involve thousands of components, making algorithm efficiency critical. We classify problems into complexity classes:

- **Class P:** Problems solvable in polynomial time $O(n^k)$ by a deterministic Turing machine. Example: Shortest path routing.
- **Class NP:** Decision problems whose solutions are verifiable in polynomial time by a deterministic machine (or solvable in polynomial time by a non-deterministic Turing machine).
- **NP-Complete:** The hardest problems in Class NP. If a polynomial-time algorithm exists for any NP-complete problem, then $P = NP$. Example: Partitioning, Floorplanning, and Routing.
- **NP-Hard:** Problems at least as hard as any NP-complete problem, but not necessarily in NP (often optimization problems).

Since most physical design steps (partitioning, placement, routing) are NP-hard optimization problems, VLSI CAD tools must use **heuristic algorithms** (like simulated annealing, genetic algorithms) to find near-optimal solutions in reasonable computing time.

2.4 Data Structures & Algorithms in VLSI CAD

VLSI design automation tools utilize classic computer science data structures and algorithms to model, analyze, and optimize physical circuits:

Concept	VLSI Application / Example	Core Role in Automation
Graphs & Hypergraphs	Netlists, routing grids, horizontal/vertical constraint graphs.	Models pin connectivity and track ordering.
Adjacency Matrix/List	Grid representation, component connection lists.	Back-end database for topological analysis.
Trees (Steiner / Slicing)	Global routing trees, slicing floorplan blocks, BDDs.	Optimizes wirelength, layout structures, and Boolean functions.
Priority Queues (Heaps)	Time-wheel event simulation, list scheduling priority queues.	Tracks next scheduled simulation events or operations.
Dijkstra / BFS / DFS	Maze routing (Lee's algorithm), reachability analysis.	Finds shortest routing paths around layout obstacles.
Graph Coloring	Register binding / allocation during HLS.	Assigns registers to non-overlapping variable lifetimes.

3. Combinatorial Optimization & Heuristic Methods

3.1 1. Simulated Annealing (SA)

Simulated Annealing is a stochastic global optimization technique inspired by the process of physical annealing in metallurgy, where a metal is heated to a high temperature and cooled slowly to reach a low-energy, highly ordered crystalline state.

Simulated Annealing Core Principle

To escape local minima, Simulated Annealing occasionally accepts worse moves (uphill steps) with a probability that decreases as the temperature decreases. This probability is governed by the Boltzmann distribution:

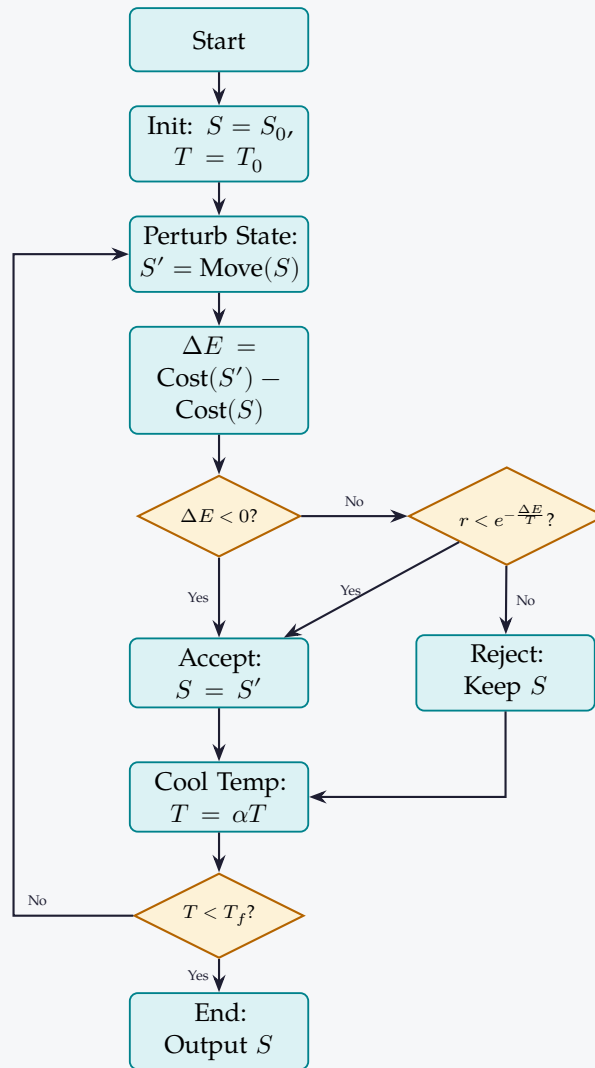
$$P(\text{accept}) = e^{-\frac{\Delta E}{T}} \quad (1)$$

where $\Delta E = \text{Cost}(S_{\text{new}}) - \text{Cost}(S_{\text{current}})$ is the change in cost, and T is the current temperature.

3.1.1 Algorithmic Flow

1. **Initialization:** Start with an initial configuration S_0 and a high initial temperature T_0 .
2. **Perturbation:** Make a small random change to the configuration to generate a neighbor state S' .
3. **Cost Evaluation:** Compute $\Delta E = \text{Cost}(S') - \text{Cost}(S)$.
4. **Acceptance Criteria:**
 - If $\Delta E < 0$ (improvement), accept the new state ($S \leftarrow S'$).
 - If $\Delta E \geq 0$ (worse state), accept S' only if a random number $r \in [0, 1)$ satisfies $r < e^{-\frac{\Delta E}{T}}$. Otherwise, reject it and keep S .
5. **Cooling:** Reduce the temperature using a cooling function (typically $T_{k+1} = \alpha T_k$, where $0.85 \leq \alpha \leq 0.98$).
6. **Termination:** Repeat until T reaches the stopping temperature T_f or the system freezes.

Simulated Annealing Flow Chart



3.2 2. Genetic Algorithms (GA)

Genetic Algorithms are population-based heuristics inspired by natural selection and evolutionary biology.

- **Representation:** Designs (e.g., placements) are encoded as strings of symbols called chromosomes.
- **Evaluation:** A fitness function determines the quality of each chromosome (e.g., inverse of total wirelength).
- **Crossover:** Pairs of chromosomes (parents) are combined with a probability P_c to create offspring.
- **Mutation:** Random bits in the offspring chromosomes are inverted with a small probability P_m to maintain genetic diversity and prevent premature convergence.
- **Selection:** Fitter individuals are selected to form the next generation.

3.3 3. Integer Linear Programming (ILP)

ILP is a mathematical optimization method where both the objective function and the constraints are linear equations, and the decision variables are restricted to integers.

- Used for exact placement, routing track allocation, and resource scheduling.
- Yields the global optimum but suffers from high computational complexity (NP-complete), making it practical only for small design subproblems.

Quick Revision — Module 1

Introduction to VLSI Design Automation | PE-EC802C

Key Formulas and Metrics at a Glance

Boltzmann Probability	$P(\text{accept}) = e^{-\frac{\Delta E}{T}}$ (accepts uphill moves when T is high)
Geometric Cooling Schedule	$T_{k+1} = \alpha T_k$ ($0.85 \leq \alpha \leq 0.98$)
Degree Sum Theorem	$\sum_{v \in V} \deg(v) = 2 E $ (Handshaking Lemma)

One-Line Definitions

- **Y-Chart:** A representation of hardware design along Behavioral, Structural, and Physical axes.
- **Heuristic:** An approximate algorithm designed to solve NP-hard optimization problems in polynomial time.
- **Bipartite Graph:** A graph whose vertices are split into two disjoint sets, with edges only running between them.
- **Tractability:** The property of a problem indicating whether it can be solved within polynomial time.
- **Crossover:** The genetic algorithm operator that merges features of two parent chromosomes to create offspring.
- **Compaction:** The physical design phase that squeezes layouts to minimum area without violating design rules.

Frequently Asked / Viva Questions

Q1. Why is physical design automation necessary in modern chip design?

Modern chips contain billions of transistors. Manual design is impossible due to complexity, human error, and time-to-market constraints. Automation ensures optimization, design rule compliance, and correct-by-construction layout generation within a few days.

Q2. What are the typical cost components optimized during cell placement?

The placement phase optimizes: (1) Total wirelength (usually estimated using half-perimeter wirelength or HPWL), (2) Chip bounding box area, (3) Routing congestion, and (4) Critical path timing delays.

Q3. Explain the metallurgical analogy used in Simulated Annealing.

In physical annealing, metal is heated to melt bonds (high energy, random atom states) and cooled very slowly. Atoms settle into a stable crystal lattice

Frequently Asked / Viva Questions (contd.)

(minimum energy state). In Simulated Annealing, the configuration represents atom positions, cost represents system energy, and the control parameter T acts as the temperature. High T allows random moves, while slow cooling lets the algorithm settle into a global minimum cost state.

Q4. What is the difference between NP-complete and NP-hard problems?

NP-complete problems belong to class NP (their solutions can be verified in polynomial time) and are at least as hard as any other problem in NP. NP-hard problems are not necessarily in NP (verification might not run in polynomial time), but any NP-complete problem can be reduced to them in polynomial time.

Q5. Why can't we use exact methods like Integer Linear Programming (ILP) for whole-chip routing?

ILP is an NP-complete problem. The execution time of ILP solvers increases exponentially with the number of integer variables. Since modern chips contain millions of nets, solving a single global ILP formulation would take years. Hence, heuristics are used instead.

Q6. How does the mutation parameter prevent premature convergence in Genetic Algorithms?

Premature convergence occurs when the population becomes uniform, trapped in a local minimum. Mutation randomly alters individual chromosome bits, reintroducing lost genetic traits and letting the algorithm explore untested search space regions.

Q7. What is a hypergraph, and how is it used in circuit partitioning?

A hypergraph is a generalization of a graph where an edge (called a hyperedge) can connect any number of vertices. In VLSI, standard cells are vertices, and multi-terminal nets connecting three or more pins are represented as hyperedges.

Q8. What role does the parameter α play in Simulated Annealing?

The parameter α is the cooling rate. If α is too small (e.g. 0.5), the temperature falls too rapidly, causing the system to freeze into a poor local minimum. If α is too close to 1 (e.g. 0.999), the cooling is extremely slow, and the algorithm takes too long to run. Standard values are between 0.85 and 0.98.

Q9. What is the difference between top-down and bottom-up design methodologies?

Top-down starts with a high-level specification and refines it down to RTL, gate-level, and finally transistor layout. Bottom-up starts by designing individual custom circuits, standard cells, or sub-blocks, and then joins them to build the complete system.

Q10. Why does routing require two separate steps: Global Routing and Detailed Routing?

Routing is highly complex. Solving it in one step is computationally impossible. Global routing simplifies the problem by dividing the chip into a coarse grid

Frequently Asked / Viva Questions (contd.)

and routing nets through grid regions without assigning specific tracks. Detailed routing then focuses on each grid region separately to assign nets to precise metal tracks and layers.

Comparison Snapshot**Structural vs. Behavioral Domain:**

Feature	Behavioral Domain	Structural Domain
Description	Focuses on algorithmic functionality and logic transitions.	Focuses on hardware components and connectivity netlists.
Example	Transfer functions, state equations, Boolean expressions.	Logic gates, flip-flops, ALUs, multiplexers.
Representation	VHDL/Verilog process blocks, C/C++ behavior models.	Structural netlists, schematic diagrams.

Class P vs. Class NP Problems:

Feature	Class P	Class NP
Solvability	Solvable in polynomial time by a deterministic machine.	Solvable in polynomial time by a non-deterministic machine.
Verification	Can be solved and verified in polynomial time.	Solutions can be verified in polynomial time.
VLSI Example	Shortest-path routing (Dijkstra's).	Cell placement, partition minimization, floorplan sizing.

Module 2

Layout Compaction

1. Layout Compaction

Layout Compaction

Layout Compaction is the process of compressing a VLSI layout in the horizontal and/or vertical directions to minimize the total chip area while ensuring that all geometric features continue to satisfy the fabrication technology's design rules (e.g., minimum spacing and minimum width rules).

1.1 1D vs. 2D Compaction

- **1D Compaction:** Squeezes the layout in one direction at a time (first X, then Y, or vice versa). It is computationally simple ($O(n \log n)$ or $O(n^2)$) but can yield suboptimal results because vertical and horizontal constraints are decoupled.
- **2D Compaction:** Squeezes the layout in both X and Y directions simultaneously. It finds the absolute minimum area but is NP-complete, making it impractical for large layouts.

1.2 Constraint Graph Compaction

A layout is modeled as a set of features (contacts, wires, gates) whose coordinates are variables. Spacing constraints between feature i and feature j are represented as:

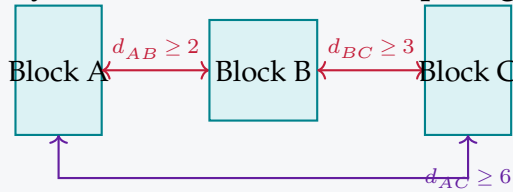
$$x_j - x_i \geq d_{ij} \quad (2)$$

where x_i, x_j are coordinates and d_{ij} is the minimum spacing requirement plus the widths of features.

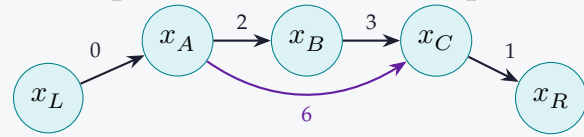
- **Graph Construction:** Vertices represent layout elements (coordinates). Directed edges represent constraints. An edge $e(i, j)$ with weight d_{ij} represents the constraint $x_j \geq x_i + d_{ij}$.
- **Longest Path formulation:** To find the minimum coordinates, we solve the longest path problem from a virtual source node ($x_{\text{left}} = 0$) using the Liao-Sarrafzadeh or Bellman-Ford algorithms.

Layout Compaction Spacing Constraints

Layout Blocks & Minimum Spacing:



Equivalent Constraint Graph:



2. Circuit Placement

2.1 1. Force-Directed Placement

Force-Directed Placement models the placement problem as a physical system of springs (Hooke's law). Standard cells are point masses, and interconnects are springs.

- **Spring Force Formula:** The force between cell i and cell j connected by a net with weight w_{ij} is:

$$F_{ij} = w_{ij}(x_j - x_i) \quad (3)$$

- **Equilibrium Equations:** The total force on cell i must be zero:

$$\sum_j w_{ij}(x_j - x_i) = 0 \quad \Rightarrow \quad x_i \sum_j w_{ij} - \sum_j w_{ij}x_j = 0 \quad (4)$$

This is written in matrix form as $Cx = f$, where C is the connectivity Laplacian matrix. Solving this system gives optimal cell coordinates that minimize quadratic wirelength.

2.2 2. Min-Cut Placement (Breuer's Algorithm)

Min-Cut placement uses a top-down divide-and-conquer strategy:

1. Start with the entire chip area containing all standard cells.
2. Bisect the chip area horizontally or vertically.
3. Use a graph partitioning algorithm (e.g. Kernighan-Lin) to partition the cells into two subsets, minimizing the number of nets crossing the cut line.
4. Recursively apply this bisection to the sub-regions until each region contains only one or a few cells.

3. Circuit Partitioning

Circuit Partitioning

Circuit Partitioning is the process of dividing a circuit (modeled as a graph or hypergraph) into two or more sub-circuits (partitions) such that the size of each partition is balanced (respecting capacity limits) and the number of connections between the partitions (the cut size) is minimized.

3.1 Circuit Representation

Before applying partitioning or placement algorithms, the physical netlist must be modeled mathematically. The common circuit representation models are:

- **Standard Graph** ($G = (V, E)$): Vertices V represent electronic components (gates, standard cells), and edges E represent electrical connections. While simple, graphs are limited because they can only model two-terminal nets.
- **Hypergraph** ($H = (V, E_h)$): A generalization of a graph where a hyperedge $e \in E_h$ can connect any subset of vertices. Hypergraphs are the most accurate model for circuits because actual nets often connect three or more pins.
- **Connectivity Matrix** (C): An $n \times n$ symmetric matrix where the entry c_{ij} represents the connectivity strength (number of common nets) between component i and component j .
- **Laplacian Matrix** (L): Formulated as $L = D - C$, where D is the diagonal degree matrix ($d_{ii} = \sum_j c_{ij}$). The Laplacian matrix is central to analytical, force-directed placement algorithms, converting wirelength minimization into a system of linear equations ($Lx = f$).

3.2 Kernighan-Lin (KL) Algorithm

The KL algorithm is an iterative, heuristic bipartitioning algorithm for undirected graphs. It starts with an initial balanced partition (A, B) of size n each ($|V| = 2n$), and swaps pairs of nodes to reduce the cut cost.

3.2.1 Mathematical Formulations

For each node $a \in A$:

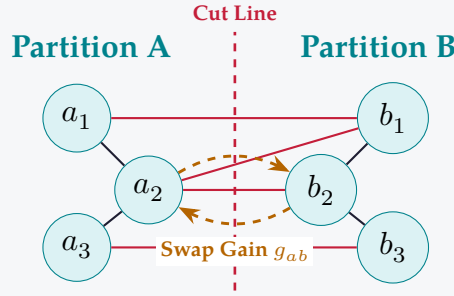
- **External Cost** (E_a): Connection cost to nodes in the other partition: $E_a = \sum_{y \in B} c_{ay}$.
- **Internal Cost** (I_a): Connection cost to nodes in the same partition: $I_a = \sum_{x \in A} c_{ax}$.
- **D-Value** (D_a): Difference representing the priority of node a for swapping:

$$D_a = E_a - I_a \quad (5)$$

- **Gain** (g_{ab}): The net reduction in cut cost if node $a \in A$ and $b \in B$ are swapped:

$$g_{ab} = D_a + D_b - 2c_{ab} \quad (6)$$

where c_{ab} is the weight of the edge between a and b .

Kernighan-Lin Swapping Concept

Swap gain formula: $g_{ab} = D_a + D_b - 2c_{ab}$. If swapped, the cut size decreases by g_{ab} .

3.2.2 Algorithmic Steps for One Pass

1. Mark all vertices as *unlocked*.
2. Compute D -values for all vertices in A and B .
3. For $i = 1$ to n :
 - Choose an unlocked pair $(a_i, b_i) \in A \times B$ that maximizes $g_i = D_{a_i} + D_{b_i} - 2c_{a_i b_i}$.
 - Lock a_i and b_i (they cannot be swapped again in this pass).
 - Update D -values of remaining unlocked neighbors:

$$D'_x = D_x + 2c_{xa_i} - 2c_{xb_i} \quad \text{for } x \in A$$

$$D'_y = D_y + 2c_{yb_i} - 2c_{ya_i} \quad \text{for } y \in B$$
4. Find the step k that maximizes the cumulative gain $G_k = \sum_{i=1}^k g_i$.
5. If $G_k > 0$, swap the sets $\{a_1, \dots, a_k\}$ and $\{b_1, \dots, b_k\}$ permanently and start a new pass. If $G_k \leq 0$, terminate (no further improvement possible).

Kernighan-Lin Step-by-Step Example**Problem:**

Consider a graph with $V = \{1, 2, 3, 4, 5, 6\}$ and edges $E = \{(1, 2), (1, 3), (1, 4), (2, 5), (3, 6), (4, 5), (5, 6)\}$. The initial partitions are $A = \{1, 2, 3\}$ and $B = \{4, 5, 6\}$. Find the gains and update the partitions.

Solution:**Step 1: Compute initial D-values:**

- Node 1: $E_1 = c_{14} = 1$; $I_1 = c_{12} + c_{13} = 2$. $\Rightarrow D_1 = 1 - 2 = -1$.
- Node 2: $E_2 = c_{25} = 1$; $I_2 = c_{21} = 1$. $\Rightarrow D_2 = 1 - 1 = 0$.
- Node 3: $E_3 = c_{36} = 1$; $I_3 = c_{31} = 1$. $\Rightarrow D_3 = 1 - 1 = 0$.
- Node 4: $E_4 = c_{41} = 1$; $I_4 = c_{45} = 1$. $\Rightarrow D_4 = 1 - 1 = 0$.
- Node 5: $E_5 = c_{52} = 1$; $I_5 = c_{54} + c_{56} = 2$. $\Rightarrow D_5 = 1 - 2 = -1$.
- Node 6: $E_6 = c_{63} = 1$; $I_6 = c_{65} = 1$. $\Rightarrow D_6 = 1 - 1 = 0$.

Step 2: Calculate gains $g_{ij} = D_i + D_j - 2c_{ij}$ for unlocked pairs:

- $g_{14} = D_1 + D_4 - 2c_{14} = -1 + 0 - 2(1) = -3$
- $g_{15} = D_1 + D_5 - 2c_{15} = -1 - 1 - 0 = -2$
- $g_{24} = D_2 + D_4 - 2c_{24} = 0 + 0 - 0 = 0$
- $g_{25} = D_2 + D_5 - 2c_{25} = 0 - 1 - 2(1) = -3$
- $g_{36} = D_3 + D_6 - 2c_{36} = 0 + 0 - 2(1) = -2 \dots$

The maximum gain is $g_1 = g_{24} = 0$ (swap Node 2 and Node 4). Lock Node 2 and Node 4.

Step 3: Update D-values for remaining unlocked nodes {1, 3} and {5, 6}:

- $D'_1 = D_1 + 2c_{12} - 2c_{14} = -1 + 2(1) - 2(1) = -1$
- $D'_3 = D_3 + 2c_{32} - 2c_{34} = 0 + 0 - 0 = 0$
- $D'_5 = D_5 + 2c_{54} - 2c_{52} = -1 + 2(1) - 2(1) = -1$
- $D'_6 = D_6 + 2c_{64} - 2c_{62} = 0 + 0 - 0 = 0$

Calculate gains for the second step:

- $g_{15} = D'_1 + D'_5 - 2c_{15} = -1 - 1 - 0 = -2$
- $g_{16} = D'_1 + D'_6 - 2c_{16} = -1 + 0 - 0 = -1$
- $g_{35} = D'_3 + D'_5 - 2c_{35} = 0 - 1 - 0 = -1$
- $g_{36} = D'_3 + D'_6 - 2c_{36} = 0 + 0 - 2(1) = -2$

Choose swap pair (1, 6) or (3, 5) with $g_2 = -1$. Let's select (3, 5) to swap. Lock {3, 5}. After calculating the remaining gain, we determine that cumulative gain G_k is not positive. The optimal choice is to not make any swaps, or restart with a different initial partition.

3.3 Fiduccia-Mattheyses (FM) Partitioning Algorithm

The FM algorithm is an extension of the KL algorithm that partitions hypergraphs. Key enhancements over KL:

- **Single Node Moves:** Instead of swapping pairs, it moves one cell at a time from its partition to another. This allows handling partitions of unequal sizes.
- **Linear Time Complexity:** Time complexity is reduced to $O(|E|)$ per pass by using a bucket array data structure to select the unlocked cell with the maximum gain in $O(1)$ time.

Quick Revision — Module 2

Partitioning, Placement & Compaction | PE-EC802C

Key Formulas and Metrics at a Glance

Constraint Spacing	$x_j - x_i \geq d_{ij}$ (directed edge $v_i \rightarrow v_j$ with weight d_{ij})
Force-Directed Coordinate	$x_i = \frac{\sum_j w_{ij} x_j}{\sum_j w_{ij}}$ (equilibrium position)
Kernighan-Lin Gain	$g_{ab} = D_a + D_b - 2c_{ab}$ ($D_k = E_k - I_k$ node priority)

One-Line Definitions

- **1D Compaction:** Compressing elements along one axis in a single step (either horizontal or vertical).
- **Constraint Graph:** A directed graph representing layout elements as vertices and spacing rules as weighted edges.
- **Hypergraph:** A graph where a hyperedge can connect any number of vertices, used to model multi-terminal nets.
- **Laplacian Matrix:** A matrix representation of connectivity used in force-directed placements.
- **Cut Size:** The number of nets or edges crossing the boundaries separating different partitions.
- **Gain (KL):** The net reduction in the cut cost resulting from swapping two nodes between partitions.
- **FM Algorithm:** A linear-time hypergraph partitioning algorithm that moves a single node at a time.

Frequently Asked / Viva Questions

Q1. What are spacing and width rules in layout compaction?

Width rules dictate the minimum dimension of a layout wire or layer (to prevent breaks due to line width tolerances). Spacing rules define the minimum distance between two separate features (to prevent short circuits due to lithographic misalignment).

Q2. How is a layout feature mapped to a constraint graph during compaction?

Each vertical layout edge (for X-compaction) or horizontal layout edge (for Y-compaction) is represented as a vertex. The minimum design rule spacing between the two edges is represented as a directed, weighted edge connecting the vertices.

Frequently Asked / Viva Questions (contd.)**Q3. What is the difference between a graph and a hypergraph?**

In a standard graph, each edge connects exactly two vertices. In a hypergraph, an edge (hyperedge) can connect a subset containing any number of vertices. Circuits are modeled as hypergraphs because a single signal line (net) typically connects multiple component pins.

Q4. Why does the KL partitioning algorithm lock swapped nodes?

If swapped nodes were not locked, the algorithm could end up in an infinite loop by swapping the same nodes back and forth. Locking them guarantees that every node is swapped at most once per pass, ensuring termination in exactly n steps.

Q5. What is the purpose of the virtual source and sink nodes in constraint graphs?

The virtual source node (x_{left}) represents the left edge of the chip boundary ($x = 0$). The virtual sink node (x_{right}) represents the right boundary. The longest path from source to sink determines the minimum width of the entire compacted layout.

Q6. How does force-directed placement handle cell overlaps?

Force-directed placement minimizes total quadratic wirelength, which pulls all connected cells toward the center of the chip, causing significant cell overlaps. To resolve overlaps, placement tools follow the force calculation step with slot assignment or recursive partitioning.

Q7. Explain the role of the bucket array structure in the FM algorithm.

The FM algorithm uses a bucket array to index unlocked cells by their gain values. Cells with the same gain are linked together. This data structure allows the cell with the maximum gain to be found and retrieved in $O(1)$ constant time, enabling overall linear time complexity.

Q8. What are the limitations of the KL algorithm?

(1) It only works on standard graphs, not hypergraphs. (2) It can only handle partitions of equal sizes. (3) Its time complexity is $O(n^3)$ per pass, which is too slow for modern circuits containing millions of nodes.

Q9. Why is quadratic wirelength used in force-directed placement instead of linear wirelength?

Quadratic wirelength uses a squared distance metric (d^2), which can be formulated as a set of linear equations. Solving linear equations is mathematically straightforward and fast (using matrix solvers). Linear wirelength optimization requires absolute values ($|d|$), which cannot be solved directly in a single step.

Q10. What is a connectivity matrix, and how does it represent net weights?

A connectivity matrix C is a matrix where the entry C_{ij} represents the number of nets connecting cell i and cell j . Higher values of C_{ij} indicate a stronger connection, which generates a stronger attractive force during force-directed placement, pulling the cells closer together.

Comparison Snapshot**1D vs. 2D Compaction:**

Feature	1D Compaction	2D Compaction
Squeezing Axis	Squeezes sequentially (X first, then Y).	Squeezes simultaneously in both X and Y.
Complexity	Polynomial time ($O(n \log n)$), very fast.	NP-complete, computationally intractable.
Optimality	Suboptimal (leaves redundant space).	Achieves absolute minimum silicon area.

KL vs. FM Partitioning:

Feature	Kernighan-Lin (KL) Algorithm	Fiduccia-Mattheyses (FM) Algorithm
Circuit Model	Standard graph (edges connect two nodes).	Hypergraph (hyperedges connect multiple nodes).
Move Strategy	Swaps pairs of nodes ($a \in A, b \in B$).	Moves a single node at a time.
Time Complexity	$O(n^3)$ per pass.	$O(E)$ (linear time) per pass.
Partition Size	Forces strictly equal sizes.	Supports flexible/unbalanced partition sizes.

Module 3

Circuit Floorplanning

1. Circuit Floorplanning

Circuit Floorplanning

Floorplanning is the process of identifying the physical shapes, aspect ratios, and positions of the major functional blocks (macros) on the silicon die. It optimizes the chip area and estimates the total wirelength prior to the cell placement phase.

1.1 Slicing vs. Non-Slicing Floorplans

- **Slicing Floorplan:** A floorplan that can be obtained by recursively partitioning a rectangle into two smaller rectangles using either a horizontal or vertical cut line.
- **Non-Slicing Floorplan:** A floorplan that cannot be partitioned using recursive bisection lines (contains cyclical structures like a wheel configuration).

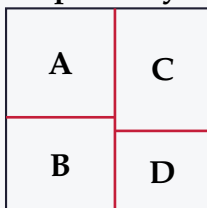
1.2 Polish Expressions and Slicing Trees

Slicing floorplans are represented using two equivalent models:

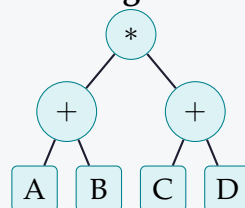
- **Slicing Tree:** A binary tree where the leaf nodes represent the blocks ($A, B, C, \dots$) and the internal nodes represent the cuts ($+$ for horizontal, $*$ for vertical).
- **Polish Expression:** A postfix representation of the slicing tree. A Polish expression is *normalized* if it contains no consecutive identical operators (e.g., $AB++$ is normalized to $AB+C+$).

Slicing Floorplan, Tree, and Polish Expression

Floorplan Layout:



Slicing Tree:



Polish Expression:
 $AB+CD+*$

- **Left Sub:** $AB+$
- **Right Sub:** $CD+$
- **Merged by:** $*$

1.3 Shape Functions and Sizing Optimization

The shape function of a block describes its width w as a function of height h .

- **For a Horizontal Cut (+) of blocks 1 and 2:**

$$w = \max(w_1, w_2), \quad h = h_1 + h_2 \quad (7)$$

- **For a Vertical Cut (*) of blocks 1 and 2:**

$$w = w_1 + w_2, \quad h = \max(h_1, h_2) \quad (8)$$

Sizing algorithms recursively combine child shape functions at internal tree nodes using horizontal or vertical addition rules, choosing the combination that minimizes total layout area.

1.4 Non-Slicing Floorplan Representations

For non-slicing floorplans (which contain wheel-like cyclic configurations that cannot be cut recursively), binary trees and Polish expressions are insufficient. Common representation models include:

- **Sequence Pair (SP):** Represents a floorplan using a pair of block permutations, e.g., (α, β) . The spatial relation between any two blocks X and Y is defined by their relative order in the two permutations:
 - $(X \text{ is before } Y \text{ in } \alpha) \text{ and } (X \text{ is before } Y \text{ in } \beta) \Rightarrow X \text{ is to the left of } Y.$
 - $(X \text{ is before } Y \text{ in } \alpha) \text{ and } (X \text{ is after } Y \text{ in } \beta) \Rightarrow X \text{ is above } Y.$
- **Bounded Slicing Grid (BSG):** A grid structure of horizontal and vertical line segments that defines placement slots. It allows representing any general topological structure without restricting cuts to slicing boundaries.

2. Circuit Routing

Routing is divided into two sequential steps to manage complexity:

1. **Global Routing:** Divides the chip into a grid of routing channels, planning a coarse pathway for each net through grid regions without assigning specific wire tracks.
2. **Detailed Routing:** Assigns nets to precise metal tracks and layers within the channels planned by the global router.

2.1 Classification of Routing Problems & Methods

Routing problems are classified based on the geometrical constraints and abstraction levels:

- **Local Routing Problems:** Connecting terminal pins within a localized region.
- **Grid-based Routing:** The routing area is discretized into a uniform grid. Wires can only run along grid tracks, and vias can only be placed at grid intersections. Easy to model but consumes significant memory.

- **Gridless (Continuous) Routing:** Wires can be placed at any coordinate, restricted only by design rule boundaries (minimum spacing and width). Highly memory efficient and supports variable wire widths.
- **Area Routing:** Connecting pins across a general 2D area (often containing arbitrary blockages and obstacles). Examples include maze routers (Lee's, Soukup's) and line-probe routers (Mikami-Tabuchi, Hightower).
- **Channel Routing:** Connecting pins along two parallel rows (boundaries) using horizontal segments on one layer and vertical segments on another (detailed in Section 4).

2.2 Global Routing & Algorithms

Global routing plans the approximate path of each net to avoid congestion across the chip area. The chip is modeled as a **Grid Graph** $G_c = (V_c, E_c)$, where each vertex v_i represents a routing region (bin) and each edge e_{ij} represents the boundary between adjacent regions with a capacity constraint c_{ij} (maximum number of wires allowed to cross).

2.2.1 Key Global Routing Algorithms

1. **Rectilinear Minimum Steiner Tree (RMST):** A Steiner tree connects a set of pins by introducing auxiliary vertices called **Steiner points** to minimize total wirelength using only horizontal and vertical segments. Finding the RMST is NP-complete, so heuristics (e.g., Hwang's theorem, FLUTE) are used.
2. **Rectilinear Steiner Arborescence (RSA):** A directed Steiner tree routed from a source pin (root) to multiple sink pins such that the path from the root to any sink is a shortest rectilinear path.
3. **Sequential Routing with Rip-up and Re-route:**
 - Router routes nets one by one sequentially (e.g. using Dijkstra's algorithm).
 - Early nets take the shortest paths, causing congestion and capacity violations for later nets.
 - **Rip-up and Re-route:** Congested nets are ripped up, edge weights are increased to penalize congestion, and the nets are rerouted until all capacity constraints are satisfied.
4. **ILP-based Global Routing:** Formulates global routing as a multi-commodity flow problem to route all nets simultaneously, optimizing total wirelength subject to capacity constraints.

2.3 Lee's Maze Routing Algorithm

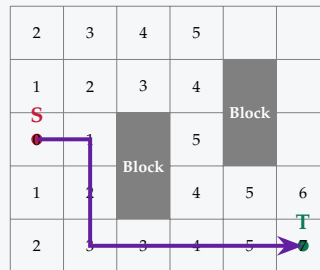
Lee's algorithm uses a Breadth-First Search (BFS) grid expansion technique to find the shortest path between a source (S) and target (T) grid cell. It guarantees finding a path if one exists and always finds the shortest path.

2.3.1 Algorithmic Flow

1. **Initialization:** Label the source cell S with 0. Set wave counter $i = 0$.

2. **Wave Expansion:** Find all unlabeled neighbor cells of cells labeled i . If a neighbor is target T , stop. Otherwise, label them $i + 1$. Repeat this expansion until the target is reached or no more cells can be labeled (blocked).
3. **Traceback:** From T , trace back to S by moving to neighbor cells with decreasing labels ($i, i - 1, \dots, 0$).
4. **Clear:** Clear all numbers on the grid.

Lee's Maze Router BFS Grid Expansion



Shortest path found: $S(0) \rightarrow (1) \rightarrow (2) \rightarrow (3) \rightarrow (4) \rightarrow (5) \rightarrow (6) \rightarrow T(7)$. Grid obstacles force the router to loop around the bottom.

2.3.2 Limitations of Lee's Router

- **High Memory Overhead:** Storing wave expansion numbers on a large grid of size $N \times N$ requires $O(N^2)$ memory.
- **Slow Execution:** Expanding wavefronts in all directions behaves like a BFS, taking $O(N^2)$ time.
- **Optimizations:** **Soukup's Router** uses a heuristic search (DFS) towards the target to speed up propagation, falling back to Lee's algorithm only when blocked. **Line-Probe Router** routes lines instead of cells, reducing memory usage.

2.4 Line-Probe Area Routing

To overcome the high memory $O(N^2)$ and execution time constraints of grid-based BFS expansion in Lee's maze router, line-probe algorithms are used:

- **Mikami-Tabuchi Algorithm:** Shoots orthogonal horizontal and vertical probe lines from both the source S and the target T . When a line generated from S intersects any line generated from T , a path is established. It runs in time proportional to the number of probe lines ($O(L)$), drastically reducing search time and memory usage.
- **Hightower's Algorithm:** Similar to Mikami-Tabuchi, but instead of shooting probes from all intersection points, it only shoots escape lines from the boundaries of obstacles obstructing the straight path. It is faster than Mikami-Tabuchi but does not guarantee finding a path even if one exists.

3. Channel Routing

Channel Routing

Channel Routing is a detailed routing technique that connects terminal pins along two parallel rows (top and bottom boundaries of a horizontal channel) using horizontal wire segments on one metal layer and vertical wire segments on a different metal layer.

3.1 Left-Edge Algorithm

The Left-Edge algorithm is a greedy track allocation algorithm that routes nets in a single pass from left to right.

1. Sort all nets in ascending order of their left-most pin x -coordinate.
2. Select the first unrouted net and assign it to the first available track.
3. Scan left to right to find the next unrouted net whose left boundary does not overlap with the previously routed net on the same track. Assign it to the same track.
4. Repeat step 3 until no more nets can fit in the current track.
5. Open a new track and repeat steps 2-4 until all nets are routed.

3.2 Horizontal and Vertical Constraint Graphs

To prevent wire overlaps, the channel router must respect two types of constraints:

- **Horizontal Constraints:** Nets overlapping in their horizontal span cannot share the same track. (Modeled using interval graphs).
- **Vertical Constraints:** If net A 's top pin is directly above net B 's bottom pin at some column, net A 's horizontal wire segment must be placed in a track above net B 's track to avoid vertical wire overlaps. (Represented as a directed **Vertical Constraint Graph (VCG)** where edge $A \rightarrow B$ means $\text{track}(A) > \text{track}(B)$).

Quick Revision — Module 3

Floorplanning & Routing | PE-EC802C

Key Formulas and Metrics at a Glance

Horizontal Cut (+)	$h_{\text{parent}} = h_1 + h_2, \quad w_{\text{parent}} = \max(w_1, w_2)$
Vertical Cut (*)	$w_{\text{parent}} = w_1 + w_2, \quad h_{\text{parent}} = \max(h_1, h_2)$
Lee's Wave Expansion	Label value $L \leq x_s - x_t + y_s - y_t $ (Manhattan limit)
Channel Routing Tracks	Lower Bound on Tracks = $\max_x d(x)$ (d_{max} density)

The maximum channel density d_{max} represents the minimum number of tracks required to route the channel without overlap.

One-Line Definitions

- **Floorplanning:** Finding optimal block locations and aspect ratios prior to cell placement.
- **Slicing Cut:** A cut line that splits a floorplan region completely into two sub-regions.
- **Polish Expression:** A postfix notation string describing slicing sequences and leaf blocks.
- **Shape Function:** A curve or step function representing possible width and height values of a block.
- **Maze Routing:** Routing algorithms that find grid-based paths between pins around layout obstacles.
- **Traceback:** The path reconstruction phase of Lee's algorithm working backward from target to source.
- **Left-Edge Algorithm:** A track allocation algorithm that routes nets sorted by their left-most coordinate.
- **Vertical Constraint Graph:** A directed graph indicating which net must be placed above another to avoid overlaps.

Frequently Asked / Viva Questions

Q1. What is a normalized Polish expression, and why is it important?

A Polish expression is normalized if it has no consecutive identical operators (e.g. $AB++$ is not normalized, whereas $AB+C+$ is). Normalization guarantees a 1-to-1 mapping between Polish expressions and slicing floorplans, which reduces the search space during optimization.

Frequently Asked / Viva Questions (contd.)**Q2. Why is floorplanning done before placement?**

Floorplanning handles large blocks (macros) and defines the global silicon structure. Placing small standard cells directly without placing macros first would lead to congestion, wiring bottlenecks, and poor cell distribution.

Q3. Explain the wave expansion step in Lee's maze routing algorithm.

Starting at the source node, neighbors are labeled with 1. Then, all unlabeled neighbors of 1 are labeled with 2. This process is repeated, labeling neighbors of i with $i + 1$, until the target cell is reached. This BFS propagation guarantees finding the shortest path.

Q4. What are the main drawbacks of Lee's maze router?

(1) It uses $O(N^2)$ memory to store the grid cell states and label numbers. (2) It is slow ($O(N^2)$ time) because the wavefront expands in all directions, looking like a growing circle, which explores many unnecessary grid nodes.

Q5. How does Soukup's router improve on Lee's algorithm?

Soukup's router uses a depth-first search (DFS) strategy to route directly toward the target cell. If it hits an obstacle, it switches to Lee's BFS expansion to find a way around the obstacle, then switches back to DFS. This reduces the search time significantly.

Q6. Explain the vertical constraint in channel routing with an example.

If Net 1 has a pin on the top row of a channel at column 3, and Net 2 has a pin on the bottom row at column 3, the vertical wire segments of Net 1 and Net 2 both lie along column 3. To prevent a short circuit, Net 1's horizontal wire segment must be placed in a track above Net 2's track.

Q7. What is a dogleg in channel routing, and why is it used?

A dogleg is a vertical wire segment that splits a net's horizontal wire segment across different tracks. It is used to resolve cyclical conflicts in Vertical Constraint Graphs, reducing the total number of tracks needed to route the channel.

Q8. How does the Left-Edge algorithm route nets?

It first sorts all nets by their left-most pin coordinates. It then processes nets in order, placing as many non-overlapping nets as possible into the first track. When no more nets can fit, it moves to the next track and repeats the process until all nets are routed.

Q9. What is the difference between slicing and non-slicing floorplans?

A slicing floorplan can be completely partitioned into individual blocks using recursive vertical and horizontal cuts. A non-slicing floorplan cannot be divided this way because it contains wheel-like cycles of blocks.

Q10. What is channel density, and why is it important?

Frequently Asked / Viva Questions (contd.)

Channel density is the maximum number of nets crossing any vertical slice of a routing channel. It represents the mathematical lower bound on the number of tracks required to route the channel without overlaps.

Comparison Snapshot**Slicing vs. Non-Slicing Floorplans:**

Feature	Slicing Floorplan	Non-Slicing Floorplan
Representation	Binary trees, Polish expressions.	Constraint graphs (BSG, Sequence Pairs).
Complexity	Optimization is fast ($O(n \log n)$).	Optimization is slow and computationally complex.
Optimal Sizing	Can be solved optimally in polynomial time.	Sizing is NP-complete.

Global Routing vs. Detailed Routing:

Feature	Global Routing	Detailed Routing
Abstraction	Coarse grid, ignores specific tracks.	Actual grid, routes on precise metal tracks.
Objective	Allocates nets to routing regions to avoid congestion.	Resolves layout design rules, placing vias and wires.
Algorithm	Steiner trees, maze routers, network flows.	Left-Edge algorithm, dogleg routers, grid maze.

Module 4

VLSI Simulation Techniques

1. VLSI Simulation Techniques

VLSI Simulation

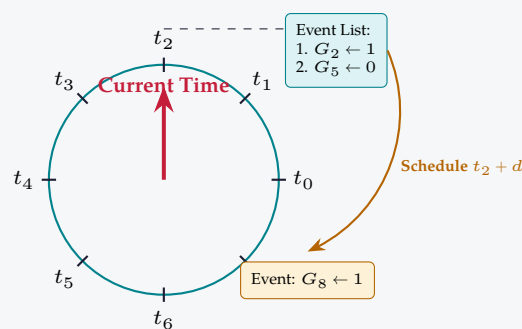
VLSI Simulation is the process of building a software model of a digital circuit and executing it with test input sequences to verify its functional correctness, check signal timings, and isolate design faults prior to physical fabrication.

1.1 Gate-Level Event-Driven Simulation

Event-driven simulators calculate logic outputs only when an input signal transition (an *event*) occurs. This is highly efficient because less than 10% of gates in a typical digital circuit switch states during any clock cycle.

- **Event Queue:** Stores pending signal updates sorted by execution time.
- **Time Wheel:** A circular queue scheduling events at future times. A pointer represents current simulation time (t). When the pointer moves to slot i , the events linked to t_i are evaluated. If a gate output changes, new events are scheduled at $t_i + d$ (where d is gate delay).

Time Wheel Scheduler Concept



1.2 Switch-Level Simulation

Models digital circuits at the transistor level by representing MOS transistors as voltage-controlled switches.

- **States:** Signals are modeled using more than two states, usually including 0 (low), 1 (high), X (unknown), and Z (high impedance).

- **Strengths:** Signals are assigned strengths to resolve bus contention (e.g. supply strength, pull strength, charge storage strength). An active pull-up transistor overrides a weak pull-down resistor.

1.3 Compiled-Code vs. Event-Driven Simulation

Gate-level simulators evaluate logic gates using two primary simulation paradigms:

- **Compiled-Code Simulation:** The digital circuit is translated into a sequence of machine instructions that are executed line-by-line. It evaluates the logic value of every gate in strict topological order during every simulation cycle, regardless of whether its input values have changed. It is highly efficient for circuits with very high switching activity, but extremely wasteful for typical circuits where only a small fraction ($< 10\%$) of gate inputs change state at any step.
- **Event-Driven Simulation:** Only evaluates a logic gate when a transition (event) occurs on at least one of its inputs. It schedules events dynamically using a priority queue (event queue or time wheel). While it avoids redundant gate evaluations, it incurs runtime overhead for queue management. It is the preferred method for large digital systems with low-to-medium active switching.

Feature	Compiled-Code Simulation	Event-Driven Simulation
Evaluation Rule	Evaluates all gates in every simulation cycle.	Evaluates gates only when input signals switch states.
Overhead	Zero scheduling or queue management overhead.	Dynamic scheduling and queue sorting overhead.
Optimal Activity	Best suited for high switching activity circuits.	Best suited for low-to-medium switching activity ($< 10\%$).
Delay Modeling	Strictly cycle-based; difficult to model gate delays.	Models detailed gate propagation and inertial delays.

2. Combinational Logic Synthesis

Logic synthesis converts high-level RTL descriptions (Verilog/VHDL) into an optimized gate-level netlist. The combinational logic synthesis process consists of two primary sequential phases:

2.1 1. Technology-Independent Optimization

Optimizes the Boolean behavior of the network without considering specific library gates. The circuit is modeled as a **Boolean Network** (a Directed Acyclic Graph where nodes represent Boolean functions and edges represent signal flow).

- **Restructuring Operations:**
 - **Decomposition:** Splitting a complex node function into smaller, simpler functions.
 - **Extraction:** Identifying common sub-expressions across multiple nodes and creating a new node to share them, reducing overall literal count.

- **Factoring:** Converting SOP forms to factored forms (e.g. $ab+ac \rightarrow a(b+c)$) to minimize transistor count.
- **Substitution:** Expressing a node's function in terms of other existing nodes.
- **Collapsing:** Merging a node into its fan-out nodes to eliminate intermediate nodes, reducing delay at the expense of area.
- **And-Inverter Graphs (AIGs):** A representation where every node is a 2-input AND gate, and edges can contain inversion flags. Highly efficient for Boolean reasoning and logic optimization.

2.2 2. Technology Mapping

Translates the optimized technology-independent Boolean network into standard library cells of the target fabrication process (e.g. static CMOS NAND, NOR, AOI gates).

- **Tree Matching (Dagon's Algorithm):**
 - The Boolean network is partitioned into a collection of trees.
 - Library gates are also represented as patterns of trees.
 - Dynamic programming is applied to find the minimum-cost cover of standard cells that matches the network tree structures.
- **Objectives:** Optimizing for minimum area, minimum delay (critical path), or minimum power dissipation.

3. Two-Level Logic Synthesis

Logic synthesis compiles high-level RTL descriptions (Verilog/VHDL) into an optimized gate-level netlist. Two-level logic synthesis targets sum-of-products (SOP) forms.

3.1 Quine-McCluskey (QM) Minimisation

An exact tabular method that guarantees finding the absolute minimum SOP expression.

1. Represent minterms in binary and group them by the number of 1s they contain.
2. Combine minterms that differ by exactly one bit (e.g., 0100 and 0101 combine to 010—), replacing the differing bit with a dash.
3. Repeat combinations until no more terms can be merged. The remaining uncombined terms are **Prime Implicants (PI)**.
4. Construct a Prime Implicant chart, identify **Essential Prime Implicants (EPI)** (columns containing only one checkmark), and select the minimum subset of PIs to cover the remaining minterms.

3.2 Espresso Heuristic Minimiser

Because QM suffers from exponential time complexity ($O(2^{2^n})$) and is unusable for more than 10 inputs, industrial synthesis tools use the Espresso heuristic optimizer. Key iterative steps in Espresso:

- **Expand:** Squeezes out variables by expanding cubes to cover adjacent minterms, reducing the total number of terms.
- **Irredundant:** Selects a minimum subset of prime cubes that covers the entire ON-set, deleting redundant terms.
- **Reduce:** Shrinks cubes to their smallest possible dimensions without uncovering any minterms, allowing the algorithm to search in new directions during the next Expand phase.

4. Binary Decision Diagrams (BDD)

Binary Decision Diagram

A **Binary Decision Diagram (BDD)** is a directed acyclic graph (DAG) representation of a Boolean function. It is constructed by recursively applying Shannon's expansion theorem.

4.1 Shannon's Expansion Theorem

Any Boolean function $f(x_1, x_2, \dots, x_n)$ can be expanded about variable x_1 as:

$$f = x_1 \cdot f_{x_1} + x_1' \cdot f_{x_1'} \quad (9)$$

where $f_{x_1} = f(1, x_2, \dots, x_n)$ is the positive cofactor, and $f_{x_1'} = f(0, x_2, \dots, x_n)$ is the negative cofactor.

Shannon's Expansion Example

Problem:

Expand the function $f = AB + B'C$ about variable A .

Solution:

1. Compute cofactors about A :

$$f_A = (1)B + B'C = B + C \quad (\text{using absorption } B + B'C = B + C)$$

$$f_{A'} = (0)B + B'C = B'C$$

2. Combine terms using Shannon's theorem:

$$f = A \cdot (B + C) + A' \cdot (B'C)$$

4.2 Reduced Ordered BDDs (ROBDD)

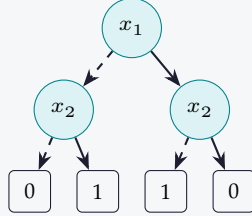
An **Ordered BDD (OBDD)** is a BDD where variables are evaluated in a fixed order along every path from root to leaf. A **Reduced OBDD (ROBDD)** is obtained by applying three reduction rules:

1. **Merge Duplicate Leaves:** Combine all leaf nodes containing 0 into a single 0-terminal, and all 1-leaves into a single 1-terminal.

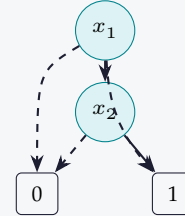
2. **Merge Duplicate Nodes:** If two internal nodes u and v represent the same variable and have identical low and high child nodes, merge them.
3. **Eliminate Redundant Nodes:** If an internal node has the same low and high child node w , delete the node and redirect all incoming edges directly to w .

BDD to ROBDD Reduction Steps

Unreduced BDD:



Reduced ROBDD (after merging):



ROBDDs are canonical representations of Boolean functions. For a fixed variable ordering, every Boolean function has exactly one unique ROBDD.

4.3 The BDD Apply Operation

The BDD **Apply** algorithm is the fundamental operation used to combine two BDDs with a binary Boolean operator (such as AND, OR, XOR). Given BDDs for f and g , the algorithm computes the BDD for $h = f \text{ op } g$ in $O(|f| \cdot |g|)$ polynomial time.

Recursive Algorithm Principle (Shannon Expansion): Let x be the variable with the highest priority (lowest index in the variable ordering) among the root variables of BDDs f and g . We can expand $f \text{ op } g$ about x as:

$$f \text{ op } g = x \cdot (f_x \text{ op } g_x) + x' \cdot (f_{x'} \text{ op } g_{x'}) \quad (10)$$

where f_x and $f_{x'}$ are cofactors of f with respect to x ($f_x = f(1)$ and $f_{x'} = f(0)$). If f does not depend on x , then $f_x = f_{x'} = f$.

Algorithmic Flow:

1. **Base Cases:** If f and g are terminal nodes (0 or 1), evaluate the operation directly (e.g., $1 \text{ AND } 0 = 0$).
2. **Memoization (Computed Table):** Before recursively expanding, check if the triple (f, g, op) has already been computed. If yes, return the cached result. This avoids exponential recursive redundancy.
3. **Recursion:** Compute the cofactors, and recursively call the Apply algorithm on the positive cofactors $h_{\text{high}} = \text{Apply}(\text{op}, f_x, g_x)$ and negative cofactors $h_{\text{low}} = \text{Apply}(\text{op}, f_{x'}, g_{x'})$.
4. **Reduction:** If $h_{\text{high}} == h_{\text{low}}$, return h_{low} (redundant node elimination). Otherwise, find or create a node for variable x pointing to h_{high} (as high child) and h_{low} (as low child), and store the result in the computed table.

Quick Revision — Module 4

Logic Synthesis, Simulation & BDDs | PE-EC802C

Key Formulas and Metrics at a Glance

Shannon's Expansion	$f = x_i \cdot f_{x_i} + x'_i \cdot f_{x'_i}$ (foundation for BDD node building)
QM Worst-Case Implicants	$N_{PI} \approx \frac{3^n}{n}$ (for n variables)
Event Scheduler Delay	$t_{\text{schedule}} = t_{\text{current}} + d_{\text{gate}}$ (event propagation delay)

One-Line Definitions

- **Time Wheel:** A circular scheduler queue used in event-driven simulators to track gate updates.
- **Switch-Level:** Transistor-level logic simulation where nodes represent strengths and states.
- **Prime Implicant:** A product term (cube) that cannot be combined with any other term to form a simpler term.
- **EPI:** A prime implicant that covers at least one minterm not covered by any other prime implicant.
- **Espresso:** An industrial heuristic logic minimizer that runs in polynomial average time.
- **Cofactor:** The Boolean function obtained by setting a specific variable to 0 or 1.
- **ROBDD:** A canonical directed acyclic graph representing a Boolean function, minimized using reduction rules.
- **Canonical Form:** A unique representation of a mathematical object, allowing easy comparison.

Frequently Asked / Viva Questions

Q1. Explain the advantage of event-driven simulation over compiler-driven simulation.

Compiler-driven simulation evaluates every gate in the circuit during every time step, which is slow. Event-driven simulation only evaluates gates whose inputs undergo a transition (an event). Since only 2 – 10% of gates change states in a clock cycle, event-driven simulation is 10 times faster.

Q2. Why are multiple strengths used in switch-level simulators?

Transistors can have different dimensions, driving strengths, or functions (e.g.

Frequently Asked / Viva Questions (contd.)

pull-up vs. pull-down). Using strengths allows the simulator to resolve contentions when multiple transistors drive the same bus line, ensuring that the stronger driver dominates.

Q3. What is the difference between Quine-McCluskey (QM) and Espresso?

QM is an exact minimization algorithm that guarantees the absolute minimum SOP form but has exponential time complexity. Espresso is a heuristic optimizer that uses Expand, Reduce, and Irredundant steps to find a near-optimal solution in polynomial average time, making it suitable for large circuits.

Q4. Why is variable ordering critical in ROBDDs?

The size (number of nodes) of an ROBDD is highly sensitive to the chosen variable order. A good ordering can result in a linear number of nodes ($O(n)$), while a poor ordering can make the node count grow exponentially ($O(2^n)$) for the exact same function.

Q5. What are cofactors, and how are they used in logic synthesis?

A cofactor is a simplified Boolean function obtained by setting a variable to a constant value ($f_{x_i} = f(x_i = 1)$). They are used in Shannon's expansion to recursively build BDDs and simplify logic expressions.

Q6. How does the Espresso algorithm use the "Reduce" operation to escape local minima?

The Reduce operation shrinks the size of prime cubes. This temporarily exposes some minterms, allowing the subsequent Expand step to expand cubes in different directions. This cycle helps the algorithm escape local minima.

Q7. What is a canonical representation, and why are ROBDDs canonical?

A canonical representation is a unique way of writing a mathematical object. ROBDDs are canonical because, for a fixed variable ordering, any Boolean function has exactly one unique ROBDD structure. This allows checking if two functions are identical in $O(1)$ constant time.

Q8. Explain the "Merge Duplicate Leaves" reduction rule in BDDs.

This rule combines all leaf nodes containing 0 into a single 0-terminal, and all leaves containing 1 into a single 1-terminal. This reduces the node count and creates a single converging point for all paths.

Q9. What is the function of the event queue in an event-driven simulator?

The event queue stores pending signal changes sorted by their execution times. As the simulator moves forward, it fetches events from the current time slot, evaluates the affected gates, and schedules new events at future slots.

Q10. Why do we need logic synthesis tools?

Logic synthesis automatically translates high-level functional code (Verilog/VHDL) into an optimized gate netlist. This speeds up design, guarantees logical correctness, and optimizes the gate count to meet timing and area constraints.

Comparison Snapshot**Gate-Level vs. Switch-Level Simulation:**

Feature	Gate-Level Simulator	Switch-Level Simulator
Primitive Element	Boolean logic gates (AND, OR, XOR).	MOS transistors (nodes, switches).
Execution Speed	Very fast (evaluates only on logic changes).	Slow (calculates charge flows and transistor strengths).
Accuracy	Ideal logic behavior.	Captures bi-directional pass transistor behavior.

BDD vs. ROBDD:

Feature	Binary Decision Diagram (BDD)	Reduced Ordered BDD (ROBDD)
Ordering	No fixed order of variables along paths.	Strict, identical variable ordering along all paths.
Uniqueness	Not unique (multiple representations).	Canonical (unique representation).
Leaf Nodes	Contains duplicate 0 and 1 terminal leaves.	Contains exactly one 0 and one 1 terminal.

Module 5

High-Level Synthesis (HLS) Core Concepts

1. High-Level Synthesis (HLS) Core Concepts

High-Level Synthesis

High-Level Synthesis (HLS) is the automated design process that translates an algorithmic description of behavior (written in C, C++, SystemC) into a Register Transfer Level (RTL) description (written in Verilog or VHDL) implementing a digital circuit controller and datapath.

1.1 Core Compilation Steps

The HLS process consists of three main subproblems that are closely coupled:

1. **Scheduling:** Assigning each operation in the algorithm to a specific execution control step (clock cycle) while respecting data dependencies and resource bounds.
2. **Allocation:** Selecting the types and quantities of hardware resources (e.g. 2 adders, 1 multiplier) to implement the operations, registers, and buses.
3. **Binding (Assignment):** Mapping each operation to a specific allocated functional unit (e.g., mapping multiplication v_3 to multiplier M_1) and mapping variables to specific registers.

1.2 Hardware & Architecture Models in HLS

HLS tools map behavioral code onto specific structural hardware templates. The primary target model is the **Controller-Datapath Model** (Finite State Machine with Datapath, FSM-D):

- **Datapath:** Contains functional units (ALUs, multipliers, adders) for arithmetic execution, registers/memories for data storage, and multiplexers/buses for routing data.
- **Controller:** An FSM that generates control signals (write enables, mux select lines) for each control step and reads status signals (flags, comparisons) from the datapath.

Microarchitectural models of functional units include:

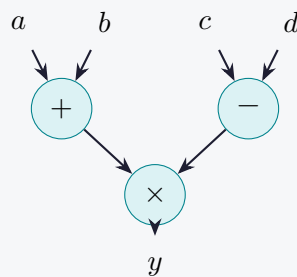
- **Combinational Units:** Complete execution within a single clock cycle (e.g. basic adders).
- **Multi-cycle Units:** Take $d \geq 2$ clock cycles to execute. No new inputs can be accepted until execution finishes (e.g. non-pipelined dividers).

- **Pipelined Units:** Take $d \geq 2$ clock cycles but can accept new inputs every II (Initiation Interval) cycles (e.g. pipelined multipliers).

1.3 Internal Representations

- **Data Flow Graph (DFG):** A directed acyclic graph where vertices represent arithmetic operations ($+$, $-$, $\times$) and directed edges represent data dependencies (values passed from one operation to another).
- **Control Data Flow Graph (CDFG):** An extension of the DFG that includes control flow structures (loops, conditional branches 'if-else', jumps). Vertices represent basic blocks of operations, and edges represent control decisions.

Data Flow Graph for $y = (a + b) \cdot (c - d)$



2. Scheduling Algorithms

Scheduling decides the overall trade-off between latency (clock cycles) and circuit area (number of functional units).

2.1 1. ASAP Scheduling (As-Soon-As-Possible)

ASAP is an unconstrained scheduling algorithm that schedules each operation to the earliest possible control step as soon as all its predecessor inputs are available.

- **Formula:** Start time $t_i = 1$ if node i has no predecessors; otherwise:

$$t_i = \max_{j \in \text{Pred}(i)} (t_j + d_j) \quad (11)$$

where d_j is the execution delay of predecessor node j (usually 1 cycle).

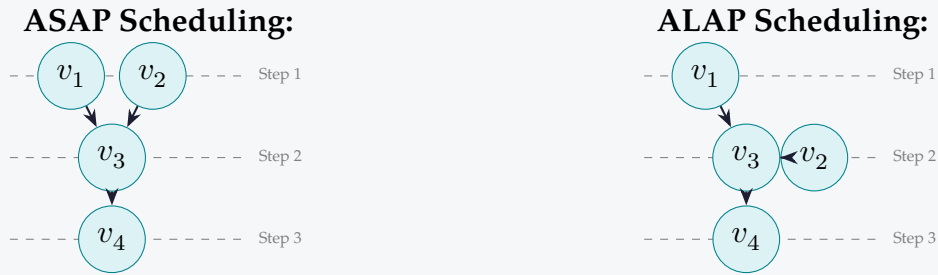
2.2 2. ALAP Scheduling (As-Late-As-Possible)

ALAP schedules each operation to the latest possible control step without violating a predefined maximum latency constraint L cycles.

- **Formula:** Start time $t_i = L$ if node i has no successors; otherwise:

$$t_i = \min_{j \in \text{Succ}(i)} (t_j) - d_i \quad (12)$$

ASAP vs. ALAP Scheduling Timelines (Latency constraint $L = 3$)



The range between ASAP and ALAP start times ($[t_{ASAP}, t_{ALAP}]$) represents the **slack** or mobility of an operation. If $t_{ASAP} = t_{ALAP}$, the operation lies on the critical path and has zero mobility.

2.3 3. List Scheduling

A resource-constrained heuristic scheduling algorithm. It resolves scheduling step-by-step using a priority queue:

1. Compute priority metrics for all operations (e.g. mobility, path length to sink).
2. For each control step $t = 1, 2, \dots$:
 - Identify the **ready list** of operations whose predecessors have completed execution.
 - Sort the ready list in descending order of priority.
 - Schedule the highest priority operations to the available functional units.
 - Defer the remaining ready operations to the next control step.

2.4 4. Force-Directed Scheduling (FDS)

FDS is a latency-constrained algorithm that minimizes resource costs by distributing operations of the same type uniformly across all control steps.

- **Probability:** The probability that operation i is scheduled in control step t is $p_i(t) = \frac{1}{\mu_i + 1}$ for $t \in [t_{ASAP}, t_{ALAP}]$, where μ_i is mobility.
- **Distribution Graph (DG):** The sum of probabilities of all operations of type op at step t : $DG(t) = \sum_i p_i(t)$.
- **Forces:** Calculates the attractive and repulsive forces between operations and control steps to place operations in slots with low distribution density, smoothing resource utilization.

3. Hardware Binding & Transformations

Binding maps variables to registers and operations to functional units.

3.1 The Assignment (Binding) Problem Formulation

The binding (assignment) problem maps operations to functional units (resource allocation constraint) and variables to registers (storage allocation constraint). It can be formulated in

different mathematical ways:

- **Graph Coloring / Max-Clique Formulation:**

- Create a **Compatibility Graph** where vertices represent operations (or variables) and edges connect vertices that can share the same resource (i.e., operations scheduled in different control steps, or variables with non-overlapping lifetimes).
- Finding a minimum clique partition in the compatibility graph (or coloring the conflict/clash graph) yields the optimal number of resources.

- **Bipartite Matching Formulation:**

- Maps operations to compatible functional units as a bipartite matching problem to minimize total steering logic (multiplexers and buses).

- **Integer Linear Programming (ILP) Formulation:**

- Let $x_{v,f} = 1$ if operation v is bound to functional unit f .
- Subject to: $\sum_f x_{v,f} = 1$ (every operation is bound to exactly one unit).
- Subject to: $x_{v_1,f} + x_{v_2,f} \leq 1$ for any two operations v_1, v_2 scheduled in the same control step (resource sharing constraint).

3.2 1. Register Binding (Left-Edge Algorithm)

Variables that do not have overlapping lifetimes can share the same hardware register.

- **Lifetime Interval:** The span between the control step where the variable is generated and the control step where it is last read.
- Left-Edge algorithm binds overlapping lifetime intervals to a minimum set of registers, identical to channel routing track allocation.

3.3 2. High-Level Optimisation Transformations

Before scheduling, HLS tools optimize the algorithmic code structure to improve performance:

- **Loop Unrolling:** Duplicates loop bodies to expose parallel operations, reducing execution latency at the cost of larger circuit area.
- **Loop Pipelining:** Overlaps the execution of subsequent loop iterations in a pipelined fashion, starting iteration $k + 1$ before iteration k finishes.
- **Tree Height Reduction:** Reorganizes expression evaluations using algebraic properties (associativity, commutativity) to balance parsing trees and reduce critical path delay.

Quick Revision — Module 5

High-Level Synthesis | PE-EC802C

Key Formulas and Metrics at a Glance

Operation Mobility	$\mu_i = t_{i,ALAP} - t_{i,ASAP}$ (slack or scheduling freedom)
ASAP Scheduling	$t_{i,ASAP} = \max_{j \in \text{Pred}(i)} (t_{j,ASAP} + d_j)$
ALAP Scheduling	$t_{i,ALAP} = \min_{k \in \text{Succ}(i)} (t_{k,ALAP}) - d_i$
FDS Operation Prob.	$p_i(t) = \frac{1}{\mu_i + 1} \quad \text{for } t \in [t_{i,ASAP}, t_{i,ALAP}]$

One-Line Definitions

- **Datapath:** The hardware block containing ALUs, multipliers, registers, and multiplexers that process data.
- **Controller:** The FSM block that generates control signals to coordinate datapath operations.
- **ASAP:** An unconstrained scheduling method that schedules operations at their earliest possible cycle.
- **ALAP:** A latency-constrained scheduling method that schedules operations at their latest possible cycle.
- **FDS:** An algorithm that minimizes functional units by distributing operations evenly across cycles.
- **Allocation:** Specifying the total hardware resources available to implement the design.
- **Binding:** Assigning logical operations and variables to specific physical functional units and registers.
- **Loop Unrolling:** A compiler optimization that replicates loop bodies to increase parallel operation opportunities.
- **Tree Height Reduction:** Reducing the depth of an expression tree using algebraic laws to speed up evaluation.

Frequently Asked / Viva Questions

Q1. What are the three main subproblems in High-Level Synthesis?

(1) Scheduling (assigning operations to clock cycles), (2) Allocation (determining the type and number of hardware resources), and (3) Binding (assigning operations and variables to specific hardware units).

Frequently Asked / Viva Questions (contd.)**Q2. Explain how register allocation can be modeled as a Graph Coloring problem.**

A variable lifetime conflict graph is constructed where vertices represent variables, and an edge connects two vertices if their lifetimes overlap. Coloring this graph such that no two adjacent vertices share the same color maps directly to register binding. The minimum number of colors corresponds to the minimum number of registers needed.

Q3. What is mobility in High-Level Synthesis scheduling, and why is it important?

Mobility is the difference between an operation's ALAP and ASAP start times. It represents the scheduling freedom (slack) of the operation. Zero mobility means the operation lies on the critical path, while high mobility allows the scheduler to shift the operation to resolve resource bottlenecks.

Q4. Explain the concept of Loop Pipelining in HLS.

Loop pipelining allows successive loop iterations to execute concurrently without waiting for the previous iteration to finish. The rate at which new iterations start is called the Initiation Interval (II). Pipelining increases throughput dramatically.

Q5. What is the difference between Resource-Constrained and Latency-Constrained scheduling?

Resource-constrained scheduling minimizes latency given a fixed limit on hardware units (e.g. List Scheduling). Latency-constrained scheduling minimizes the hardware unit cost given a fixed maximum execution time (e.g. ALAP, FDS).

Q6. How does Tree Height Reduction reduce critical path delay?

Consider evaluating $a + b + c + d$. Sequential evaluation $((a + b) + c) + d$ has a depth of 3 adder stages. Using associativity, we can compute $(a + b) + (c + d)$ in a tree format with a depth of only 2 stages, reducing execution delay.

Q7. What is functional unit binding, and what is its goal?

Functional unit binding assigns scheduled operations (e.g. additions) to specific physical components (e.g., Adder 1). The goal is to maximize sharing, which minimizes the total area of functional units, multiplexers, and interconnect buses.

Q8. Explain how the Left-Edge algorithm is used in Register Binding.

It sorts variables by the start of their lifetimes. It then assigns non-overlapping variable lifetimes to the same register track sequentially, maximizing register sharing and minimizing the total register count.

Q9. What is the role of the FSM controller in the generated RTL netlist?

The Finite State Machine (FSM) controller generates the control signals (e.g. multiplexer select lines, register write enables) at each clock cycle, coordinating the datapath to execute operations in the scheduled order.

Q10. Why is high-level synthesis preferred over writing manual RTL?

Frequently Asked / Viva Questions (contd.)

HLS allows designing at a higher level of abstraction (C/C++), which speeds up development, makes verification easier, and allows designers to quickly explore different trade-offs between area and speed by changing scheduling constraints.

Comparison Snapshot**ASAP vs. ALAP Scheduling:**

Feature	ASAP Scheduling	ALAP Scheduling
Constraints	Unconstrained (assumes infinite resources).	Latency-constrained (limited to L cycles).
Goal	Finds the minimum possible latency boundary.	Finds the latest start times to delay operations.
Search Order	Topological search starting from source nodes.	Reverse topological search starting from sink nodes.

DFG vs. CDFG:

Feature	Data Flow Graph (DFG)	Control Data Flow Graph (CDFG)
Information	Models only data dependencies and arithmetic operations.	Models both data dependencies and control decisions.
Branches/Loops	Cannot represent loops or conditional branches.	Explicitly models loop jumps and conditional branches.
Use Case	Used for basic scheduling and binding.	Used for full behavioral synthesis of systems.